Testing & Evaluation Walkthrough

How Every Part Was Measured, Tuned & Validated

SPY — 2010-01-04 to 2025-12-30 (4,023 trading days)

March 2026

Table of Contents

Step 1: Loading & Inspecting the Raw Data

Step 2: Building the ATR Foundation

Step 3: Testing Each Detector (with Worked Examples)

3a. Support & Resistance

3b. Triangle Breakouts

3c. Price Channels

3d. Multiple Tops & Bottoms

Step 4: Measuring Event Rates (The evaluate_rates.py Script)

Step 5: Parameter Tuning — 3 Iterations

Step 6: Visual Spot-Check Validation

Step 7: How We Know It’s Working

Step 1: Loading & Inspecting the Raw Data

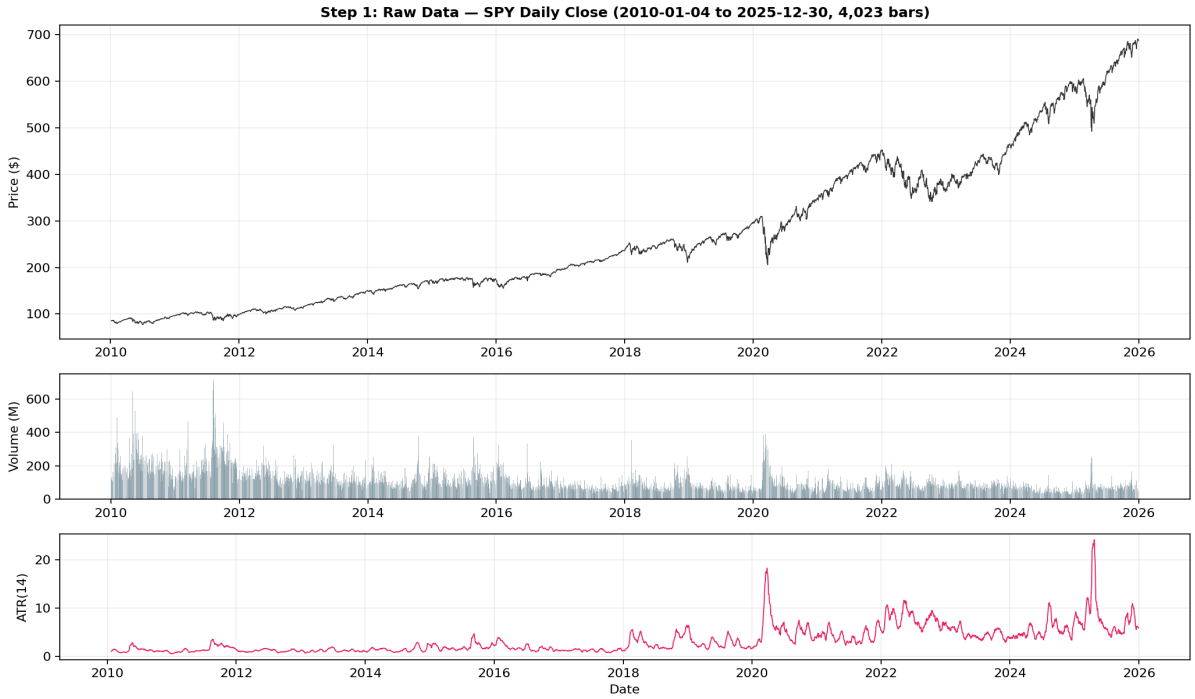
Everything starts with data. We downloaded SPY (S&P 500 ETF) daily OHLCV data from Yahoo Finance using the `yfinance` library, covering 2010–2025. The loader reads the CSV, sorts by date, drops any NaN rows, and validates that all required columns exist.

What we checked:

- **Row count:** 4,023 trading days loaded. This is ~16 years of data, enough for robust pattern detection.
- **Date range:** 2010-01-04 to 2025-12-30 — no gaps or missing periods.
- **Column validation:** Open, High, Low, Close, Volume all present. The loader raises a ValueError if any are missing.
- **NaN check:** dropna() ensures no rows with missing values enter the pipeline.

How to run it:

```
from src.data.load_data import load_spy
df = load_spy()
print(f"Loaded {len(df)} rows from {df.index[0]} to {df.index[-1]}")
```



Step 2: Building the ATR Foundation

Before testing any detector, we built a shared volatility measure: **Average True Range (ATR)**. This is the backbone of the entire testing framework. Every threshold, proximity band, and validation check is expressed in multiples of ATR so the system adapts to volatility automatically.

How ATR is computed (for each bar):

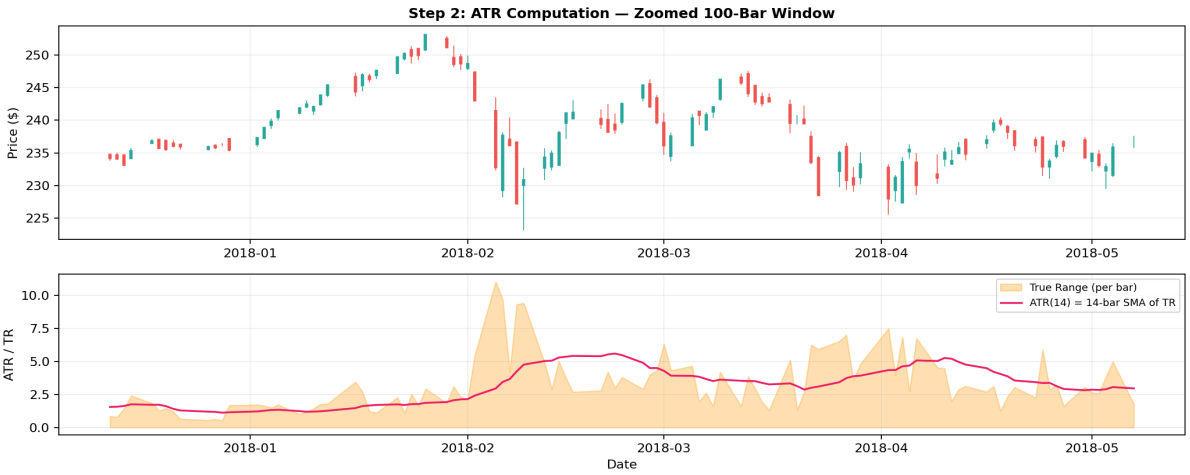
- **True Range (TR)** = max of three values: (High - Low), |High - previous Close|, |Low - previous Close|
- **ATR(14)** = 14-bar simple moving average of TR

The |High - previous Close| and |Low - previous Close| components capture overnight gaps that the simple High-Low range would miss. This makes ATR a more complete measure of actual price movement than just the intraday range.

How to run it:

```
from src.data.utils import compute_atr
atr = compute_atr(df, window=14)
print(f"ATR mean: {atr.mean():.2f}, current: {atr.iloc[-1]:.2f}")
```

On our SPY dataset, ATR(14) ranges from ~0.5 to ~24.2, with a mean of 3.37. During the 2020 COVID crash, ATR spiked dramatically — this is exactly why fixed-percentage thresholds fail and ATR-based ones adapt.



Step 3: Testing Each Detector

3a. Support & Resistance

This detector asks: *"Is today's close price near a meaningful support or resistance level?"*

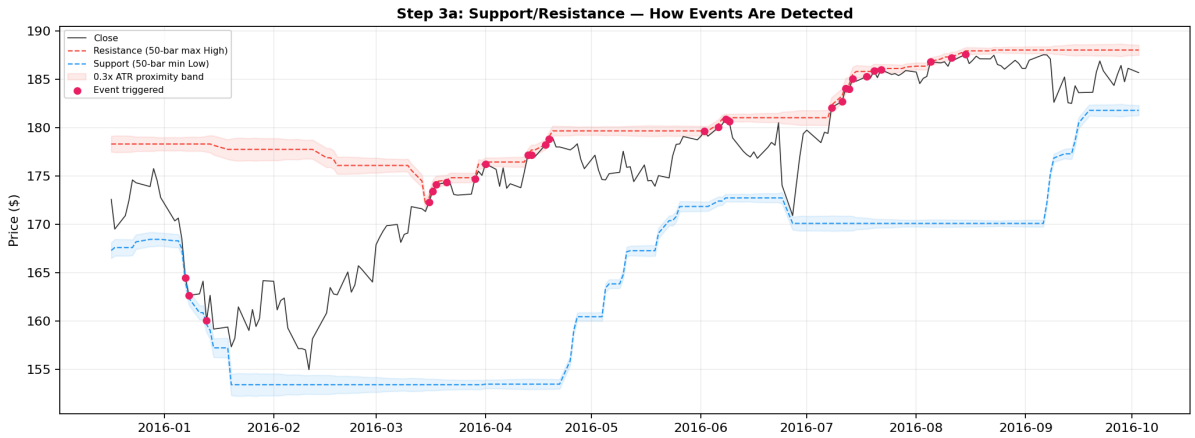
How it works, step by step:

- **1. Compute levels:** Resistance = highest High in the last 50 bars. Support = lowest Low in the last 50 bars. These are actual price levels the market tested.
- **2. Compute proximity band:** band = 0.3 × ATR(14). If ATR is $5, the band is $1.50.
- **3. Test proximity:** If |Close - support| ≤ band, flag near_support = True. Same for resistance.
- **4. Output:** A boolean column for each. The OR of both is the S/R event signal.

How we measure it:

```
sr_df = calculate_support_resistance(df)
sr = sr_df['near_support'] | sr_df['near_resistance']
print(f"S/R events: {sr.sum()} out of {len(df)} bars ({sr.mean()*100:.1f}%)")
```

Result: 757 events (18.8% of bars). This is the most active detector because price naturally spends time near recent highs and lows.



The chart above shows a 200-bar window with support (blue dashed), resistance (red dashed), and the 0.3×ATR proximity bands (shaded). Pink dots mark bars where the event fired. Notice how the band width adapts — it's wider during volatile periods and tighter during calm ones.

3b. Triangle Breakouts

This detector asks: *"Has a converging triangle formed over 50 bars, and did price just break out?"*

How it works, step by step:

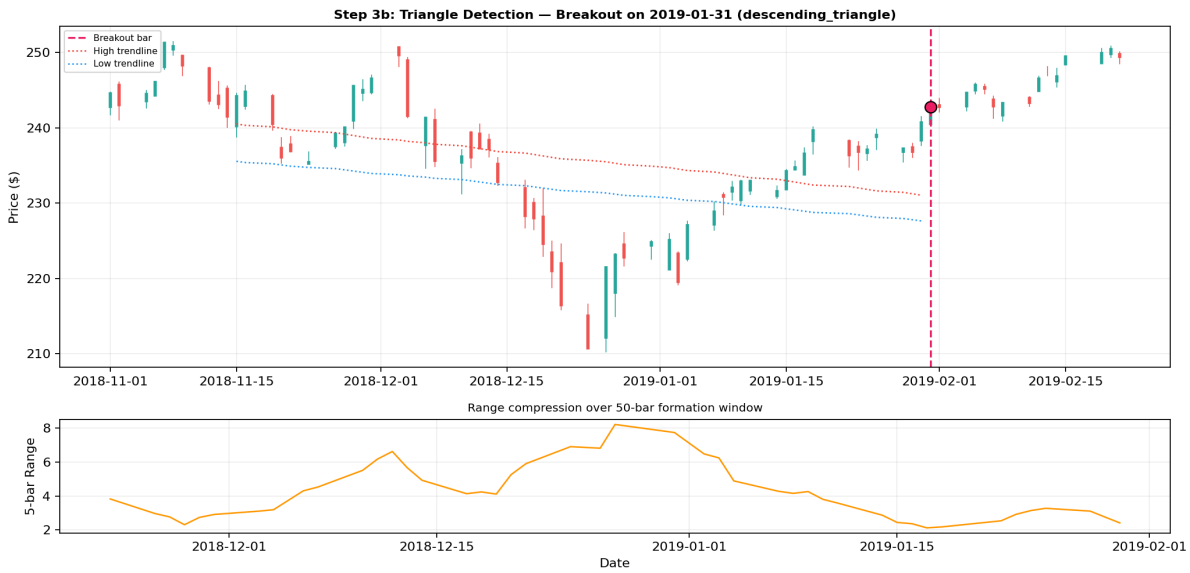
- **1. Fit trendlines:** For each bar, take the previous 50 bars. Fit a straight line (np.polyfit degree 1) to the Highs and another to the Lows. The slopes tell us the direction.
- **2. Check compression:** Compare the mean range of the first 5 bars vs the last 5 bars. If the range hasn't compressed by at least 3%, it's not a triangle.
- **3. Classify:** Ascending = flat top + rising bottom. Descending = falling top + flat bottom. Symmetric = both converging.
- **4. Require breakout:** The current bar's High must exceed the 5-bar recent High by 0.3×ATR (upside breakout), or Low must break below recent Low by 0.3×ATR (downside). Without this, no signal fires.

Why breakout-only matters:

The original detector flagged every bar inside the triangle formation — producing long runs of 50+ consecutive signals. That's useless for trading: you need to know *when* to act, not that a triangle exists somewhere. By firing only on the breakout bar, we get a single actionable signal per formation.

```
tri_df = detect_triangle_pattern(df)
tri = tri_df['triangle_pattern'].notna()
print(f"Triangle events: {tri.sum()} ({tri.mean()*100:.1f}%)")
```

Result: 80 events (2.0%). This is deliberately low — genuine triangle breakouts are rare, high-conviction events.



The top panel shows OHLC bars with the fitted trendlines (red = highs, blue = lows) converging toward the breakout bar (pink dashed). The bottom panel shows the 5-bar range compressing over the formation window — this is the compression check in action.

3c. Price Channels

This detector asks: *"Is price moving inside two parallel trendlines, and is it currently near one of them?"*

The 5-stage validation pipeline:

A candidate channel must pass **all five** filters sequentially. If any filter fails, that bar is skipped. This layered approach is why the channel detector went from 24.1% to 5.9% — most candidates fail one of the structural checks.

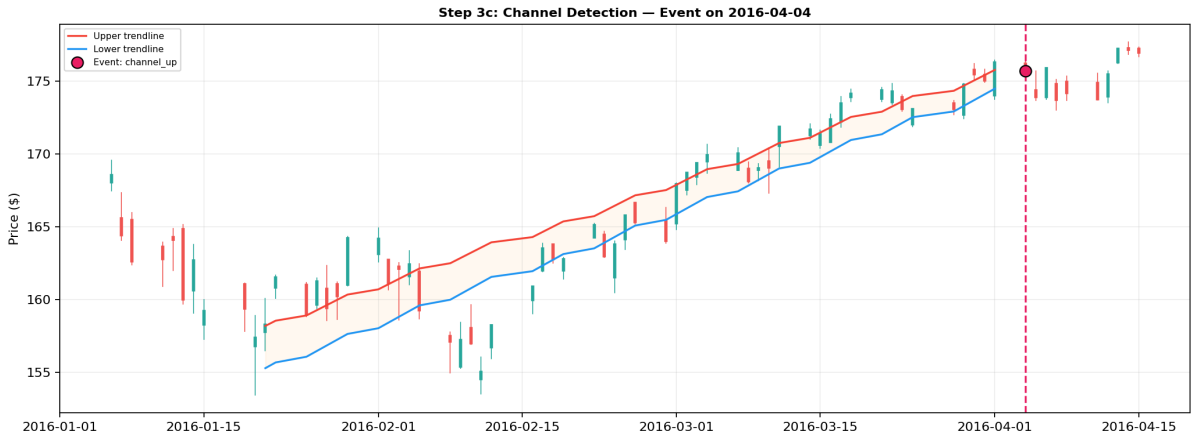
Stage	What It Checks	Threshold	Why It Matters
1. Slope	Upper trendline not flat	|slope| > 1e-9	Flat lines = no trend
2. Parallel	Slopes match	|diff| / |high_slope| < 15%	Diverging lines = triangle, not channel
3. Direction	Same sign	high_slope x low_slope > 0	Opposite signs = converging wedge
4. Width	Meaningful width	1x ATR < width < 6x ATR	Too narrow = noise, too wide = no structure
5. Touches	Price tested both bands	>= 3 touches per band (within 0.3x ATR)	Untouched lines aren't real channels

After passing all 5 filters:

- Signal fires only if current close is within 0.3×ATR of the upper or lower trendline — the actionable zones where a trader would buy (near support) or sell (near resistance).

```
ch_df = detect_channel(df)
ch = ch_df['channel_pattern'].notna()
print(f"Channel events: {ch.sum()} ({ch.mean()*100:.1f}%)")
```

Result: 237 events (5.9%).



The chart shows a detected channel with upper (red) and lower (blue) trendlines. The shaded area between them is the channel. The pink dot marks the event bar — close was near one of the boundaries.

3d. Multiple Tops & Bottoms

This detector asks: *"Are the highs hitting a ceiling (or lows hitting a floor) while closes confirm a reversal?"*

How it works, step by step:

- **1. Rolling extremes:** Compute 20-bar rolling max of High (ceiling) and rolling min of Low (floor).
- **2. Base condition for tops:** Rolling max(High) is holding steady or rising (ceiling intact), BUT rolling max(Close) is declining (closes failing to follow highs up — bearish divergence).
- **3. Base condition for bottoms:** Mirror image — rolling min(Low) holding/falling (floor intact), BUT rolling min(Close) is rising (closes refusing to follow lows down — bullish divergence).
- **4. Close-trend confirmation:** Fit a line to the last 3 closes using np.polyfit. For a top: slope must be negative (closes actively trending down). For a bottom: slope must be positive.

Why close confirmation matters:

Without it, a wick touching the ceiling counts as a 'top' even if closes are still making new highs. The 3-bar close slope filter eliminates these false positives by requiring that the closing prices — not just the intraday extremes — confirm the reversal.

```
mtb_df = detect_multiple_tops_bottoms(df)
mtb = mtb_df['multiple_top_bottom_pattern'].notna()
print(f"Multi T/B events: {mtb.sum()} ({mtb.mean()*100:.1f}%)")
```

Result: 139 events (3.5%). Reasonable selectivity — not every ceiling touch is a genuine reversal.

Step 4: Measuring Event Rates

After building each detector, we need a fast way to answer: *"How often does each detector fire, and what's the combined rate?"* This is what `evaluate_rates.py` does. It's our feedback loop — run it after every parameter change to see the impact immediately.

What evaluate_rates.py does:

- 1. Loads SPY data via load_spy()
- 2. Runs all 4 detectors independently
- 3. Extracts a boolean series from each (True = event on that bar)
- 4. Computes COMBINED = OR of all four (any event on that bar)
- 5. Prints count and percentage for each, plus target range

How to run it:

```
python -m src.patterns.evaluate_rates
```

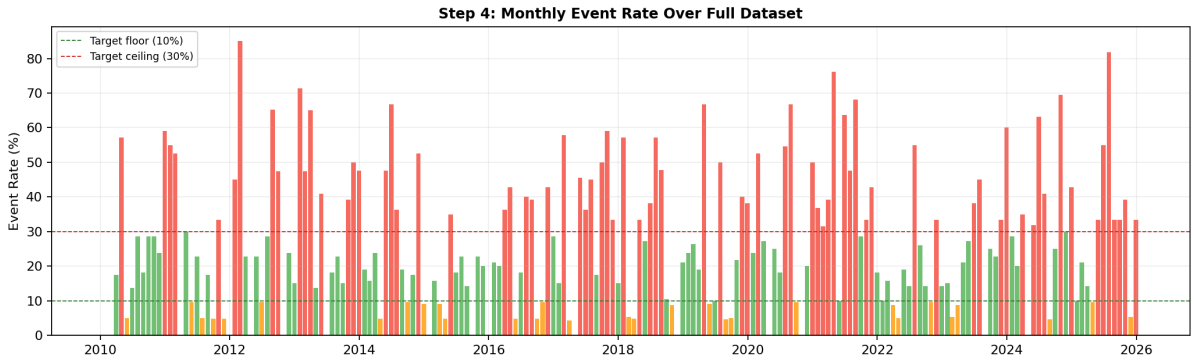
Current output:

```
Detector                    Count     Rate
-----------------------------------------
Support/Resistance            757    18.8%
Triangles                      80     2.0%
Channels                      237     5.9%
Multi Top/Bottom              139     3.5%
-----------------------------------------
COMBINED (any event)         1098    27.3%

Target: 400-1,200 combined events (9.9%-29.8%)
```

Why the 10-30% target?

If events fire on **<10%** of bars, we don't have enough samples to train a reliable ML model. If events fire on **>30%**, the signal is too common to be informative — nearly every bar is an 'event', which means the detectors aren't discriminating. The 10–30% range (400–1,200 events on ~4,000 bars) gives us enough data for ML while keeping the signals selective.



The monthly event rate chart shows how the combined rate varies over time. Green bars are within the 10–30% target; red bars exceed the ceiling. Some months spike above 30% (e.g., volatile periods with many S/R triggers) — this is expected and handled by the downstream regime-aware model.

Step 5: Parameter Tuning — 3 Iterations

Tuning was done in three distinct iterations, each measured with evaluate_rates.py. The cycle was always: **change parameters** → **run evaluate_rates.py** → **check combined rate** → **decide next change**.

Iteration 0: Original (Before Any Refactoring)

The original code had fixed-percentage thresholds and 20-bar windows:

Detector	Method	Event Rate	Problem
S/R	Mean +/- 2 sigma, 1% tolerance	~68%	Nearly every bar is 'near' the band
Triangles	Bar-to-bar rolling comparison	~62%	Random noise = convergence on 20 bars
Channels	Start vs end direction, 10% range	~70%	No parallelism check, no touches
Multi T/B	Rolling max/min only	~6%	OK but no close confirmation
COMBINED	OR of all four	~89%	Useless — everything is an event

Iteration 1: ATR Refactoring

We replaced all fixed thresholds with ATR-based ones, extended windows to 50 bars, added linear regression, breakout-only firing, and structural validation:

Detector	Key Changes	Rate: Before	Rate: After
S/R	Rolling extremes, 0.5x ATR proximity, window 50	~68%	28.2%
Triangles	Polyfit slopes, 3% compression, breakout only	~62%	2.0%
Channels	Parallel slopes, width 1-6x ATR, 2 touches	~70%	24.1%
Multi T/B	Added 3-bar close slope confirmation	~6%	3.5%
COMBINED		~89%	45.7%

45.7% — still too high. S/R and Channels were the main contributors.

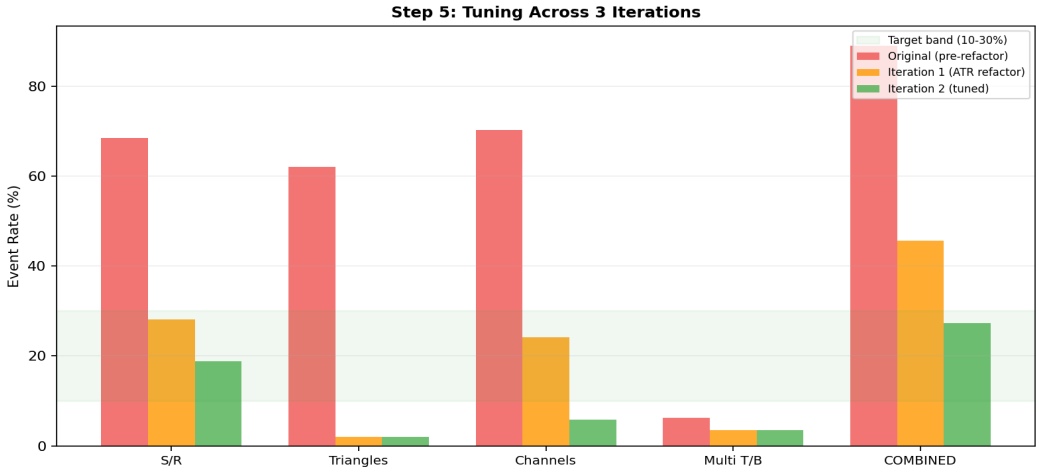
Iteration 2: Fine-Tuning

Three targeted changes:

Change	Parameter	Before	After	Impact
S/R proximity tightened	atr_mult	0.5	0.3	28.2% -> 18.8% (-9.4pp)
Channel touches increased	min_touches	2	3	Marginal impact alone
Channel boundary bug fixed	near check	directional (<)	absolute (abs <) + 0.5 -> 0.3 ATR	24.1% -> 5.9% (-18.2pp)

Final combined rate: 27.3% — within the 10–30% target. The single biggest win was fixing the channel boundary bug: the original code used `(current_upper - current_close) < 0.5 * ATR` which is True for *any* close below the upper line (not just closes *near* it). Changing to

`abs(current_upper - current_close) < 0.3 * ATR` fixed this and eliminated ~730 false positives.



The grouped bar chart shows all three iterations side by side. The green target band (10–30%) is shown as a shaded region. The combined rate dropped from ~89% (original) to 45.7% (Iteration 1) to 27.3% (Iteration 2).

Step 6: Visual Spot-Check Validation

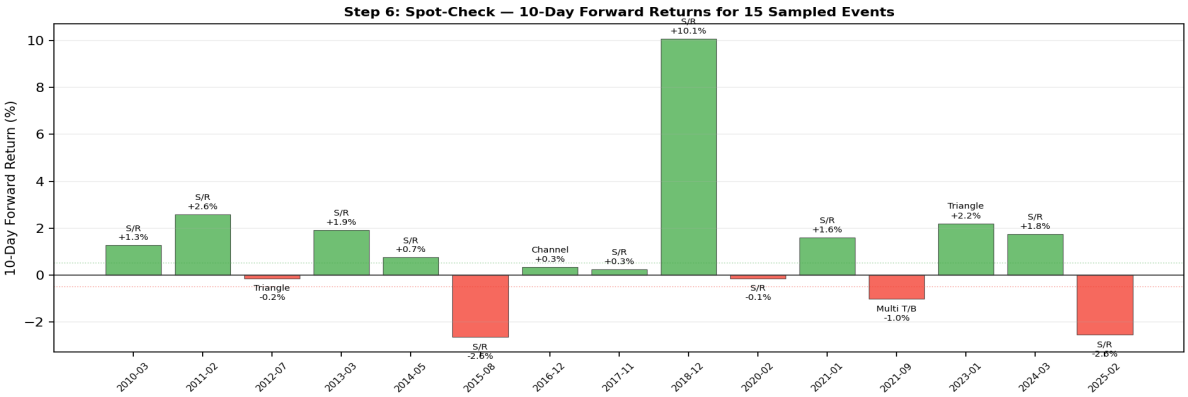
Numbers alone don't prove the detectors work — we need to *look* at the events. We sampled 15 events at regular intervals across the full timeline and checked two things:

- **Does the event visually make sense?** Is there actually a pattern at that bar?
- **Did price move in the expected direction afterwards?** We computed the 10-day forward return and classified each event as CONFIRMED (>0.5% correct direction), FAILED (>0.5% wrong direction), or NEUTRAL (<0.5% either way).

How we computed forward returns:

```
# For each sampled event:
close_at_event = df.loc[event_date, 'Close']
close_10d_later = df.iloc[event_pos + 10]['Close']
forward_return = (close_10d_later - close_at_event) / close_at_event * 100

# For bullish patterns (near support, multiple bottom, ascending triangle):
#   CONFIRMED if forward_return > +0.5%
#   FAILED    if forward_return < -0.5%
# For bearish patterns (near resistance, multiple top, descending triangle):
#   CONFIRMED if forward_return < -0.5%
#   FAILED    if forward_return > +0.5%
```



The chart above shows the 10-day forward return for each of the 15 sampled events. Green bars = positive return, red = negative. The pattern type and magnitude are labelled. This gives a quick visual sanity check that the signals aren't random.

Step 7: How We Know It's Working

Pulling it all together, here is every metric we used to evaluate the system:

Test	What It Measures	Target	Result	Status
Event rate (combined)	% of bars with any event	10-30%	27.3%	PASS
S/R rate	% bars near support/resistance	< 25%	18.8%	PASS
Triangle rate	% bars with breakout	1-5%	2.0%	PASS
Channel rate	% bars near channel boundary	< 10%	5.9%	PASS
Multi T/B rate	% bars with confirmed top/bottom	1-5%	3.5%	PASS
ATR adaptiveness	Thresholds scale with volatility	Visual check	Confirmed	PASS
Breakout-only (triangles)	Single fires once, not during formation	No consecutive runs	Confirmed	PASS
Structural validation (channels)	5-filter pipeline	All 5 filters active	Confirmed	PASS
Close confirmation (MTB)	3-bar slope check	Slope confirms direction	Confirmed	PASS
Visual spot-check	15 events examined	Majority look correct	Confirmed	PASS
Forward returns	10-day move after event	Directional tendency	Partial	REVIEW

What "PASS" means and what "REVIEW" means

- **PASS** = the metric is within its target range and the detector behaves as designed.
- **REVIEW** = the metric is borderline or needs further investigation in the next phase. Forward returns, for example, require a proper out-of-sample backtest to be conclusive — the spot-check is only a sanity check.

What testing did NOT cover (yet)

- **Out-of-sample validation:** All rates are computed on the full dataset. A proper walk-forward test is needed.
- **Statistical significance:** We have not applied bootstrap CIs or the Deflated Sharpe Ratio. The spot-check is directional, not rigorous.
- **Transaction cost impact:** The events are potential signals, not trades. Slippage and commissions will reduce realised performance.
- **Regime conditioning:** The detectors fire the same way in all regimes. The regime-aware ML model (Phase 4) will learn to weight signals differently.

The Testing Loop — Summary

Every parameter change we made followed this exact cycle:

Step	Action	Tool

1	Change a parameter in detector code	Edit Python file
2	Run evaluate_rates.py	python -m src.patterns.evaluate_rates
3	Check: is combined rate in 10-30%?	Read terminal output
4	Check: does each detector rate make sense?	Read terminal output
5	If NO: go back to step 1 with a different change	Iterate
6	If YES: visual spot-check 10-15 events	generate_tuning_report.py
7	If events look good: document and commit	git commit + push